

Week 10 Practical

Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

```
divya@Divyasree:~$ mkdir inode_project
divya@Divyasree:~$ cd inode_project
divya@Divyasree:~/inode_project$ pwd
/home/divya/inode_project
divya@Divyasree:~/inode_project$ echo "This is the original inode test file." > original.txt
divya@Divyasree:~/inode_project$ echo "This is the original inode test file." > original.txt
divya@Divyasree:~/inode_project$ cat original.txt
This is the original inode test file.
divya@Divyasree:~/inode_project$ ls -li original.txt
34648 original.txt
divya@Divyasree:~/inode_project$ stat original.txt
  File: original.txt
  Size: 38          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 34648         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   divya)   Gid: ( 1000/   divya)
Access: 2026-09-30 17:28:28.999048462 +0000
Modify: 2026-09-30 17:28:15.957457004 +0000
Change: 2026-09-30 17:28:15.957457004 +0000
 Birth: 2026-09-30 17:28:10.817457196 +0000
divya@Divyasree:~/inode_project$ ln original.txt hardlink.txt
divya@Divyasree:~/inode_project$ ls -li
total 8
34648 -rw-r--r-- 2 divya divya 38 Sep 30 17:28 hardlink.txt
34648 -rw-r--r-- 2 divya divya 38 Sep 30 17:28 original.txt
divya@Divyasree:~/inode_project$ echo "Modified through hard link." >> hardlink.txt
divya@Divyasree:~/inode_project$ cat original.txt
This is the original inode test file.
Modified through hard link.
divya@Divyasree:~/inode_project$ ln -s original.txt symlink.txt
divya@Divyasree:~/inode_project$ ls -li
```

```
34648 -rw-r--r-- 2 divya divya 66 Sep 30 17:29 hardlink.txt
34648 -rw-r--r-- 2 divya divya 66 Sep 30 17:29 original.txt
43718 lrwxrwxrwx 1 divya divya 12 Sep 30 17:29 symlink.txt -> original.txt
divya@Divyasree:~/inode_project$ stat symlink.txt
  File: symlink.txt -> original.txt
  Size: 12          Blocks: 0          IO Block: 4096   symbolic link
Device: 8,48      Inode: 43718         Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/   divya)   Gid: ( 1000/   divya)
Access: 2026-09-30 17:30:05.468095094 +0000
Modify: 2026-09-30 17:29:57.368428763 +0000
Change: 2026-09-30 17:29:57.368428763 +0000
 Birth: 2026-09-30 17:29:57.368428763 +0000
divya@Divyasree:~/inode_project$ stat -L symlink.txt
  File: symlink.txt
  Size: 66          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 34648         Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/   divya)   Gid: ( 1000/   divya)
Access: 2026-09-30 17:29:46.416524919 +0000
Modify: 2026-09-30 17:29:37.516525252 +0000
Change: 2026-09-30 17:29:37.516525252 +0000
 Birth: 2026-09-30 17:28:10.817457196 +0000
divya@Divyasree:~/inode_project$ ls -li original.txt hardlink.txt symlink.txt
34648 -rw-r--r-- 2 divya divya 66 Sep 30 17:29 hardlink.txt
34648 -rw-r--r-- 2 divya divya 66 Sep 30 17:29 original.txt
43718 lrwxrwxrwx 1 divya divya 12 Sep 30 17:29 symlink.txt -> original.txt
divya@Divyasree:~/inode_project$ ls -li original.txt
34648 original.txt
divya@Divyasree:~/inode_project$ find . -inum 1234567 -ls
divya@Divyasree:~/inode_project$ find . -type l -ls
43718 0 lrwxrwxrwx 1 divya divya 12 Sep 30 17:29 ./symlink.txt -> original.txt
divya@Divyasree:~/inode_project$ ^C
divya@Divyasree:~/inode_project$ find . -type f -ls
34648 4 -rw-r--r-- 2 divya divya 66 Sep 30 17:29 ./hardlink.txt
```

```

43718 0 lrwxrwxrwx 1 divya divya 12 Sep 30 17:29 ./symlink.txt -> original.txt
divya@Divyasree:~/inode_project$ ^C
divya@Divyasree:~/inode_project$ find . -type f -ls
34648 4 -rw-r--r-- 2 divya divya 66 Sep 30 17:29 ./hardlink.txt
34648 4 -rw-r--r-- 2 divya divya 66 Sep 30 17:29 ./original.txt
divya@Divyasree:~/inode_project$ rm hardlink.txt
divya@Divyasree:~/inode_project$ ls -li
total 4
34648 -rw-r--r-- 1 divya divya 66 Sep 30 17:29 original.txt
43718 lrwxrwxrwx 1 divya divya 12 Sep 30 17:29 symlink.txt -> original.txt
divya@Divyasree:~/inode_project$ stat original.txt
  File: original.txt
  Size: 66          Blocks: 8          IO Block: 4096   regular file
Device: 8,48   Inode: 34648       Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   divya)   Gid: ( 1000/   divya)
Access: 2026-09-30 17:29:46.416524919 +0000
Modify: 2026-09-30 17:29:37.516525252 +0000
Change: 2026-09-30 17:31:41.469792742 +0000
Birth: 2026-09-30 17:28:10.817457196 +0000
divya@Divyasree:~/inode_project$ rm original.txt
divya@Divyasree:~/inode_project$ ls -li
total 0
43718 lrwxrwxrwx 1 divya divya 12 Sep 30 17:29 symlink.txt -> original.txt
divya@Divyasree:~/inode_project$ cat symlink.txt
cat: symlink.txt: No such file or directory
divya@Divyasree:~/inode_project$ |

```

Design a program using mmap() to perform file reading and writing.
Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity

```

divya@Divyasree:~$ mkdir mmap_project
divya@Divyasree:~$ ^C
divya@Divyasree:~$ cd mmap_project
divya@Divyasree:~/mmap_project$ pwd
/home/divya/mmap_project
divya@Divyasree:~/mmap_project$ dd if=/dev/zero of=source.txt bs=1M count=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.0697814 s, 1.5 GB/s
divya@Divyasree:~/mmap_project$ ls -lh source.txt
-rw-r--r-- 1 divya divya 100M Sep 30 18:00 source.txt
divya@Divyasree:~/mmap_project$ nano traditional_io.c
divya@Divyasree:~/mmap_project$ nano mmap_io.c
divya@Divyasree:~/mmap_project$ gcc -Wall -O2 traditional_io.c -o traditional_io
divya@Divyasree:~/mmap_project$ gcc -Wall -O2 mmap_io.c -o mmap_io
divya@Divyasree:~/mmap_project$ ls -lh
total 101M
-rwxr-xr-x 1 divya divya 17K Sep 30 18:02 mmap_io
-rw-r--r-- 1 divya divya 2.2K Sep 30 18:02 mmap_io.c
-rw-r--r-- 1 divya divya 100M Sep 30 18:00 source.txt
-rwxr-xr-x 1 divya divya 16K Sep 30 18:02 traditional_io
-rw-r--r-- 1 divya divya 1.2K Sep 30 18:01 traditional_io.c
divya@Divyasree:~/mmap_project$ time ./traditional_io source.txt traditional_output.txt
File copied using read()/write().

real    0m0.095s
user    0m0.000s
sys     0m0.100s
divya@Divyasree:~/mmap_project$ time ./mmap_io source.txt mmap_output.txt
File copied using mmap().

real    0m0.123s
user    0m0.009s

```

```

sys      0m0.104s
divya@Divyasree:~/mmap_project$ ls -lh source.txt traditional_output.txt mmap_output.txt
-rw-r--r-- 1 divya divya 100M Sep 30 18:03 mmap_output.txt
-rw-r--r-- 1 divya divya 100M Sep 30 18:00 source.txt
-rw-r--r-- 1 divya divya 100M Sep 30 18:02 traditional_output.txt
divya@Divyasree:~/mmap_project$ cmp source.txt traditional_output.txt
divya@Divyasree:~/mmap_project$ for i in {1..5}; do time ./traditional_io source.txt traditional_$i.txt; done
File copied using read()/write().

real    0m0.095s
user    0m0.004s
sys     0m0.091s
File copied using read()/write().

real    0m0.110s
user    0m0.008s
sys     0m0.100s
File copied using read()/write().

real    0m0.135s
user    0m0.000s
sys     0m0.134s
File copied using read()/write().

real    0m0.127s
user    0m0.000s
sys     0m0.123s
File copied using read()/write().

real    0m0.120s
user    0m0.000s
sys     0m0.116s
divya@Divyasree:~/mmap_project$ |

```

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[]) {

    if (argc != 3) {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;

    // Open source file
    src = open(argv[1], O_RDONLY);

    if (src == -1) {
        perror("Error opening source file");
        return 1;
    }

    // Create destination file
    dest = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

```

```

    perror("Error creating destination file");
    close(src);
    return 1;
}

// Copy using read() and write()
while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0) {

    if (write(dest, buffer, bytes_read) != bytes_read) {
        perror("Error writing file");
        close(src);
        close(dest);
        return 1;
    }
}

if (bytes_read == -1) {
    perror("Error reading file");
}

close(src);
close(dest);

printf("File copied using read()/write().\n");

return 0;
}

```

```

GNU nano 7.2                                     mmap_io.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main(int argc, char *argv[]) {

    if (argc != 3) {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    int src, dest;
    struct stat st;
    void *src_map;
    void *dest_map;

    // Open source file
    src = open(argv[1], O_RDONLY);

    if (src == -1) {
        perror("Error opening source file");
        return 1;
    }

    // Get file size

```

[Read 104 lines]

```

    close(src);
    return 1;
}

size_t file_size = st.st_size;

// Create destination file
dest = open(argv[2], O_RDWR | O_CREAT | O_TRUNC, 0644);

if (dest == -1) {
    perror("Error creating destination file");
    close(src);
    return 1;
}

// Set destination file size
if (ftruncate(dest, file_size) == -1) {
    perror("ftruncate");
    close(src);
    close(dest);
    return 1;
}

// Map source file into memory
src_map = mmap(NULL,
               file_size,
               PROT_READ,
               MAP_PRIVATE,
               src,

```

```

               0);

if (dest_map == MAP_FAILED) {
    perror("mmap destination");
    munmap(src_map, file_size);
    close(src);
    close(dest);
    return 1;
}

// Copy data through memory
memcpy(dest_map, src_map, file_size);

// Make sure changes are written to disk
if (msync(dest_map, file_size, MS_SYNC) == -1) {
    perror("msync");
}

// Unmap files
munmap(src_map, file_size);
munmap(dest_map, file_size);

close(src);
close(dest);

printf("File copied using mmap().\n");

return 0;
}

```